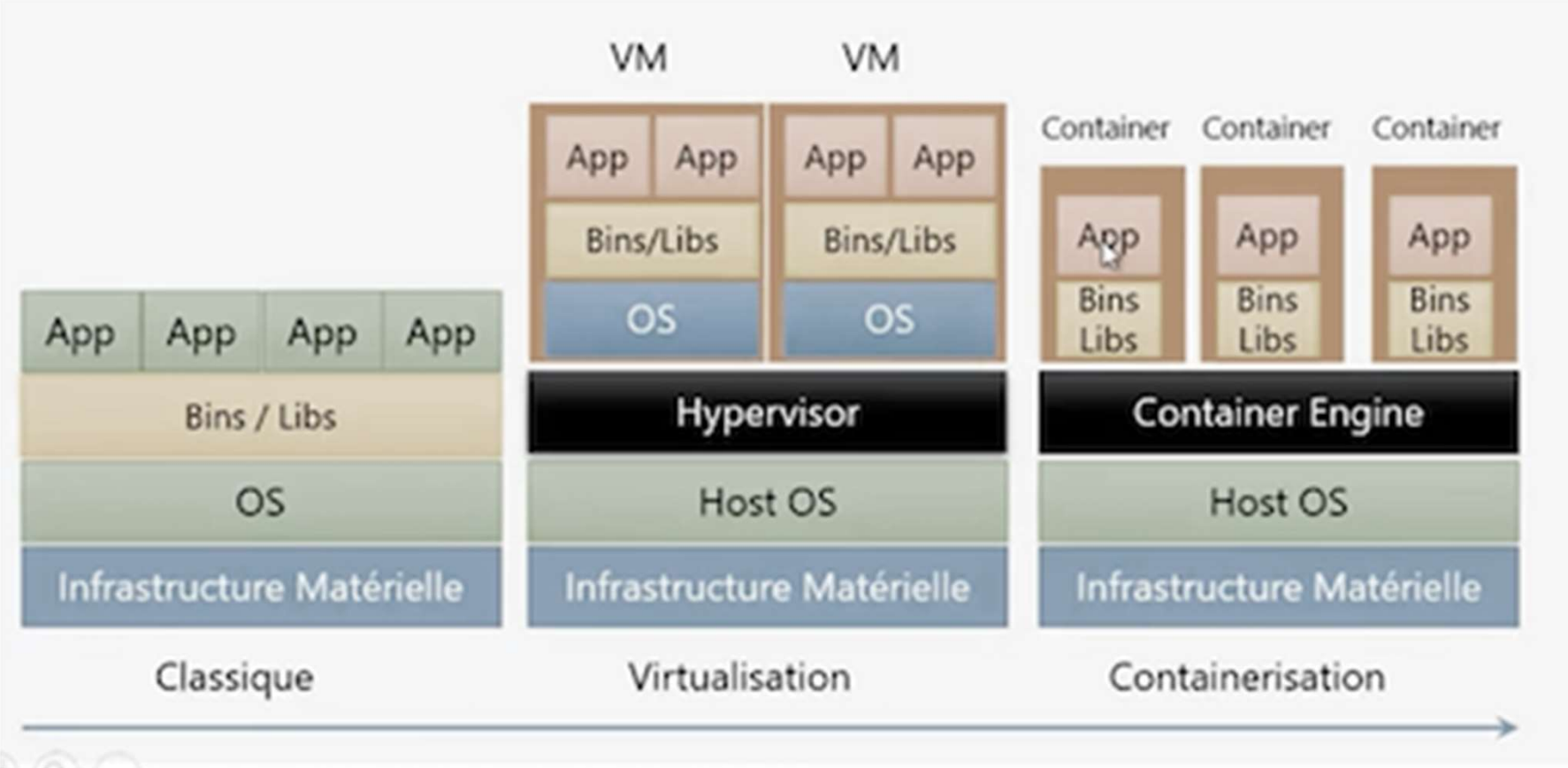
Containerisation

Containerisation

Evolution de l'infrastructure

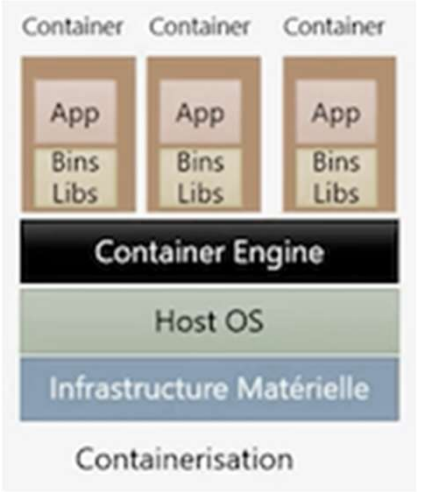


Conteneurisation

C’est quoi un conteneur?

- Enveloppe permettant de packager une application avec juste ce dont elle a besoin pour fonctionner
- Peut être déployé tel quel dans n'importe quelle machine disposant d'un **Container Engine** avec différents environnement (Dev, Test, Prod)
- Utilise le Kernel de l'OS Hôte
- A son propre espace de processus et sa propre interface réseau
- Isolé de l'hôte, mais exécutée directement dessus.
- Permet de décomposer l'infrastructure applicative en petits éléments légers facile à déployer et à réutiliser



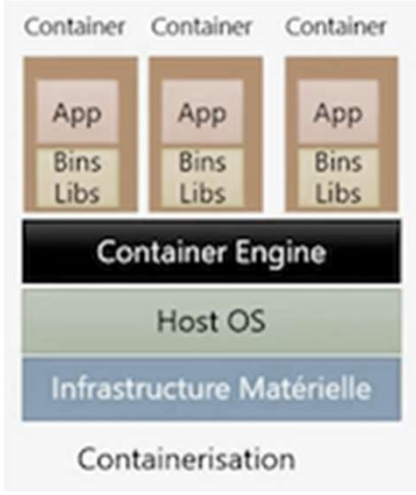


Conteneurisation

Avantages du conteneur

- Meilleures performances que les VM (Démarrage instantané)
- Portabilité d'un environnement à l'autre (Multicloud)
- Cohérence entre les environnements Dev, test et prod
- Permet de modulariser facilement l'application
- Gérer l'héritage technique (Ancienne application) grâce à l'isolation

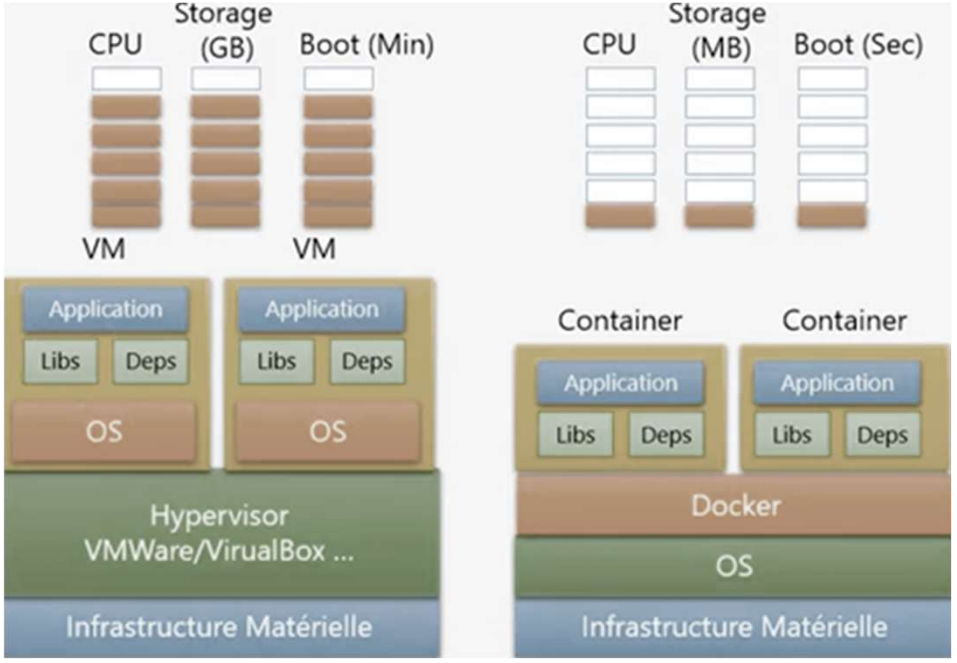




Conteneurisation

Conteneurs vs Machine Virtuelles

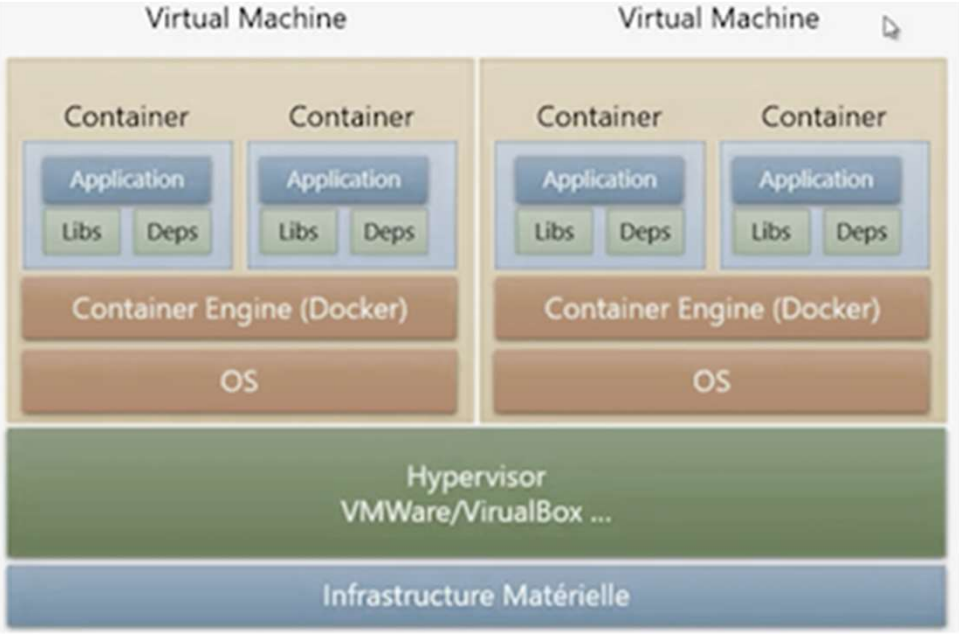
- Machine Virtuelle:
  - Permet de virtualiser une machine physique
  - Chaque VM a son propre OS
  - Une VM consomme beaucoup de ressources (CPU, Stockage) et prend assez de temps pour booter (qq minutes).
- Conteneur:
  - Permet de créer un environnement d'exécution des applications
  - Les conteneurs utilisent le même OS
  - Tous les conteneurs utilisent le même kernel OS (Linux), consomme peu de ressources, boot rapide (qq secondes)



Conteneurisation

Utiliser des conteneurs dans des machine virtuelles

- Les container engine ne viennent pas pour remplacer les machines virtuelles.
- Dans la pratique on utilise les deux:
  - Les machines virtuelles pour virtualiser les machines
  - Utiliser un container engine tel que Docker pour isoler les environnements d'exécution des applications dans des machines virtuelles.
- Ceci pour tirer le bénéfice des deux technologies



Conteneurisation

Historique

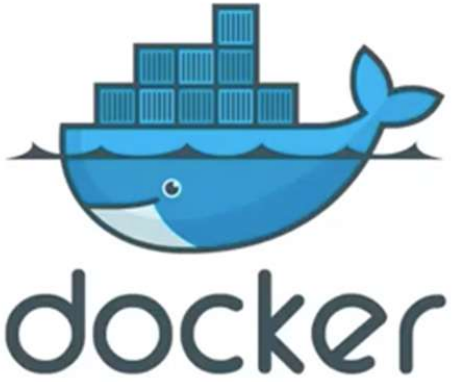
- Conteneur = unité de transport intermodal
- Virtualisation de processus :
  - UNIX chroot (1979-1982)
  - BSD Jail (1998)
  - Parallels Virtuozzo (2001)
  - Solaris Containers (2005)
  - Linux LXC (2008)
  - **Docker (2013) (basé sur LXC)**



Conteneurisation avec Docker

C’est quoi docker?

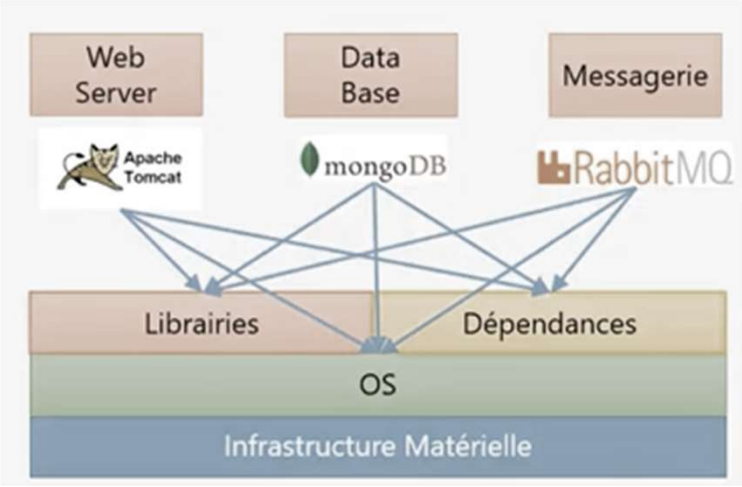
- Docker permet de créer des environnements (appelées containers) de manière à isoler des applications.
- Il permet d'empaqueter une application ainsi que les dépendances nécessaires dans un conteneur virtuel isolé qui pourra être exécuté sur n'importe quelle machine supportant docker.
- Docker est un logiciel libre qui permet le déploiement d'applications sous la forme de conteneurs logiciels.
- L'origine de docker est :
  - Au départ, société française et maintenant basée à San Francisco.
  - dotCloud : un PaaS avec un container engine écrit en Python
  - En 2012 : Réécriture le langage Go.
  - En 2013, Première version Open source de docker
  - Réaction très positive de la communauté
  - dotCloud change de nom pour Docker.
  - En 2014, Levée de fonds : 40 millions $
  - En 2015, Levée de fonds : 95 millions $



Conteneurisation avec Docker

Problèmes de déploiement des applications

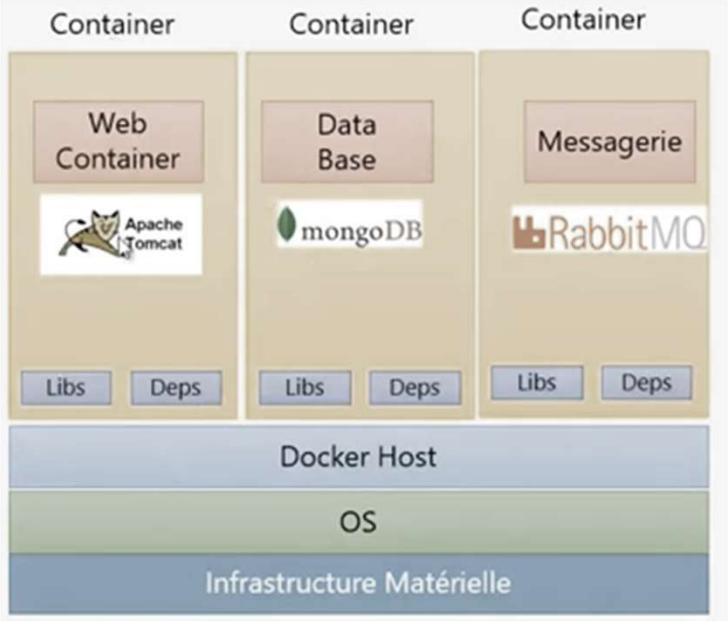
- Pour une application utilisant différentes technologies (services), des problèmes sont posés au moment du déploiement en production:
  - Compatibilité des applications avec les OS
  - Installer les dépendances et les librairies requises avec les bonnes versions pour chaque service.
  - Installer les différents environnements :
    - Dev
    - Test
    - Prod
- Ce qui prend beaucoup de temps pour déployer les applications.
- Avec des conflits entres les développeurs et les opérationnels (Administrateurs systèmes)



Conteneurisation avec Docker

Problèmes de déploiement des applications

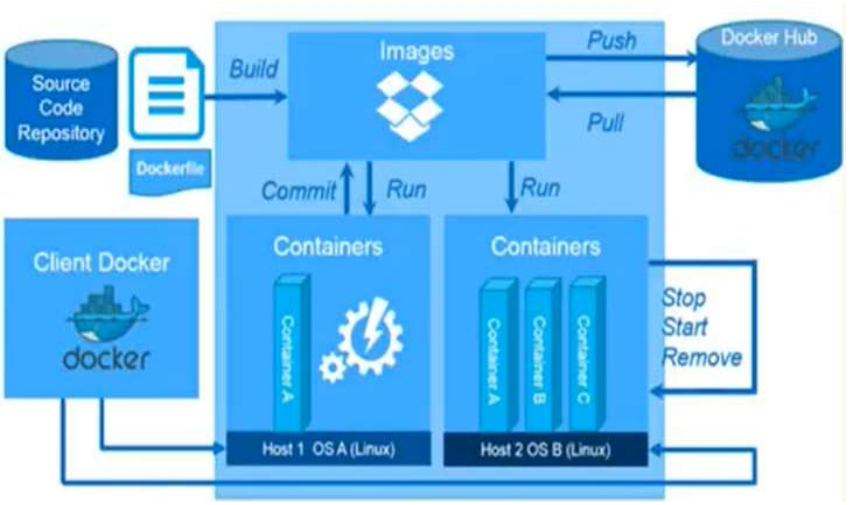
- Solution: conteneurs d'applications
  - Embarquer les applications dans des conteneurs
  - Exécuter chaque service avec ses propres dépendances dans des conteneurs séparées



Conteneurisation avec Docker

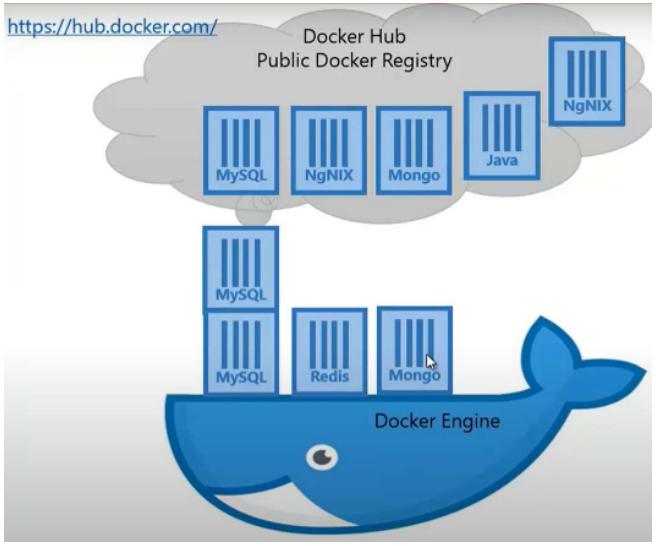
Architecture globale de docker

- Le développeur crée un fichier **Dockerfile** contenant les commandes que docker va exécuter pour construire une image docker de cette application.
  - **$ docker build**
- L'image docker contient tout ce dont l'application a besoin pour s'exécuter correctement.
- Les images Docker peuvent être publiées dans un registre public (Docker hub) ou privé.
  - **$ docker push image_name**
- Pour télécharger une image docker d'une application dans un Host Docker, il suffit d'utiliser:
  - **$ docker pull image_name**
- La création et l'exécution d'un conteneur d'une application se fait par instanciation et exécution de l'image en utilisant :
  - **$ docker run image_name**
- Avec docker run, si l'image n'existe pas dans le host, elle va procéder au téléchargement celle-ci avant d'en créer et exécuter un conteneur docker.
- Docker se compose de :
  - Docker Engine qui permet de créer le Host Docker sur une machine Linux (Docker daemon)
  - Un client Docker qui peut se trouver dans n'importe quelle autre machine et qui est connecté à Docker Engine via différents connecteurs exposés par docker (socket, REST API, etc.)



Conteneurisation avec Docker

Docker hub




```
$ docker run mysql
$ docker run redis
$ docker run mongodb
$ docker run mysql
```

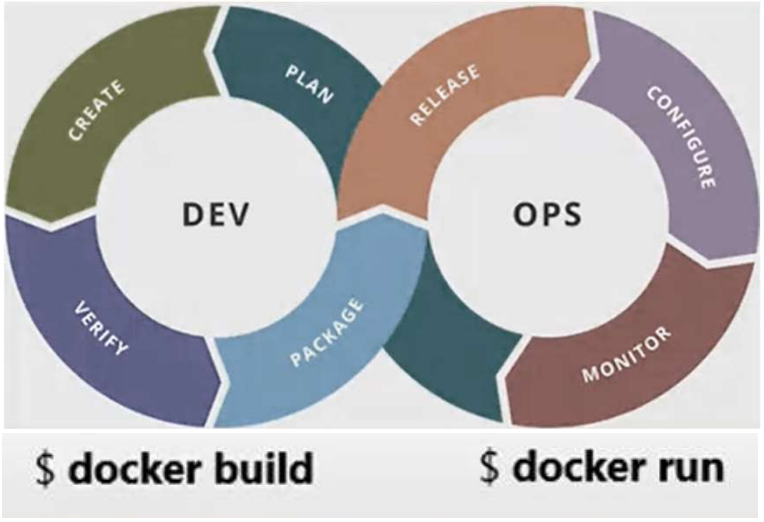
- Une image docker est juste un fichier package représentant la Template des conteneurs. Elle définit la structure du conteneur en englobant l'application containérisée et l'ensemble de ses dépendances.

- Un Conteneur représente une instance d'une image. Un conteneur est exécuté par le Docker Host. Ce qui implique l'exécution de l'application qu'il transporte dans un environnement isolé fourni par le conteneur.

Conteneurisation avec Docker

Docker contribue à instaurer l'approche devops

- **Sans Docker :**
  - Le développeur
    - Développe l'application.
    - Génère le package de l'application à déployer (App.war)
    - Envoie à l'opérationel (Administrateur système)
      - App.war
      - Un descriptif des dépendances qu'il faut installer et configurer pour que l'application s'exécute normalement.
  - L'opérationnel
    - Doit se débrouiller pour satisfaire les exigences de l'application.
    - Pour chaque mise à jour, c'est toujours les mêmes histoires qui se répètent.
  - Ce qui rend la vie dur au administrateur systèmes (Opérationnels)
  - Ce qui crée beaucoup de conflits entre les développeurs qui tentent d'améliorer constamment les applications et les opérationnels qui doivent redéployer les mises à jour.
- **Avec Docker:**
  - Le développeur
    - Développe l'application
    - Construit une image docker de son application contenant toutes les dépendances dont l'appli a besoin.
    - Publie l'image docker dans le registre docker
  - L'opérationnel déploie l'application en instanciant des conteneurs à parti de l'image docker récupérée à partir du repository docker.





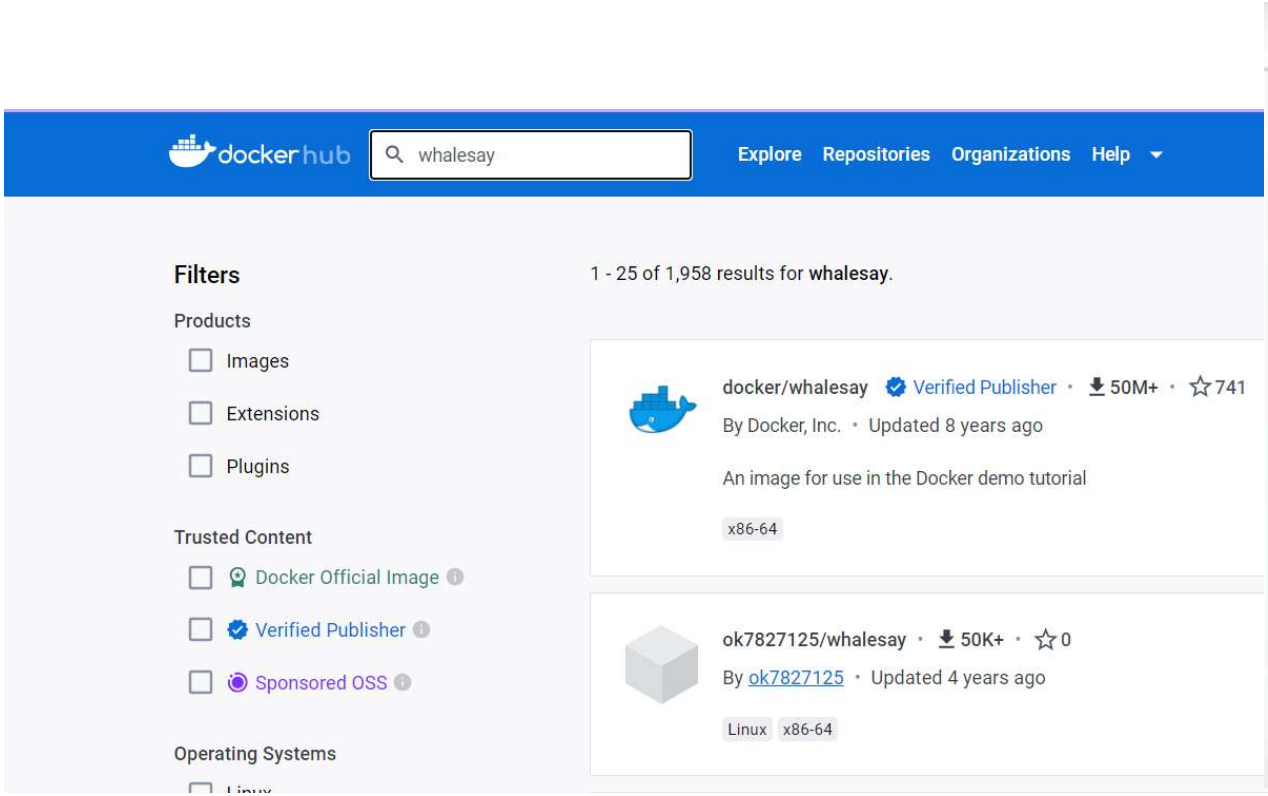
Conteneurisation avec Docker

Les éditions de Docker

- Docker est disponible en deux éditions :
- **Community Edition** (Open source)
  - Disponible pour les platforms :
    - Linux
    - Mac
    - Windows
    - Cloud (Azure, AWS,GCP)
- Pour Windows et Mac deux solutions :
  - Installer une Machine virtuelle Linux avec VirtualBox ou Vmware par exemple
  - Installer Docker desktop pour Windows Server ou Mac
- **Enterprise Edition**

Conteneurisation avec Docker

Explorer Docker hub





Conteneurisation avec Docker

Les commandes de base de docker : run et ps

- **$docker run image name**
  - Permet de créer et démarre un conteneur nouvelle instance de l'image.
  - Si l'image n'existe pas, elle sera d'abord téléchargée à partir du docker hub
- **$docker run -d image name**
  - Permet de démarrer le conteneur comme service de fond.
- La valeur par défaut de la version de l'image est **latest**
- Pour spécifier la version (tag):
  - **$docker run image name:tag**
  - Exemple : $ docker run redis: 4.0
- **$ docker ps**
  - Permet de lister les conteneur qui sont en cours d'exécution
  - Chaque conteneur créé dispose d'un identifiant unique et d'un nom du conteneur.
- **docker ps –a**
  - Permet de lister tous les conteneur avec leurs status (Up, Exited, Created)

Conteneurisation avec Docker

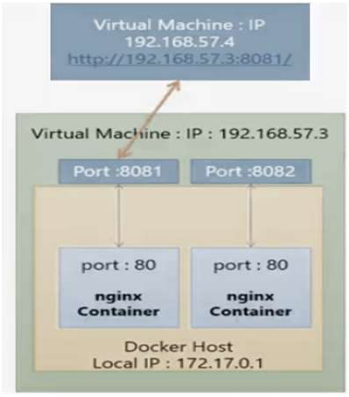
Les commandes de base de docker : run et ps

```
myubuntu@myubuntu:~$ sudo docker run nginx
[sudo] password for myubuntu:
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
a378f10b3218: Pull complete
4dfff0708538: Pull complete
2135e49ace4b: Pull complete
c843f6b280ce: Pull complete
6f35ab6f1400: Pull complete
6c538b49fa4a: Pull complete
d57731fb9008: Pull complete
Digest: sha256:b4af4f8b6470febf45dc10f564551af682a802eda1743055a7dfc8332dffa595
```

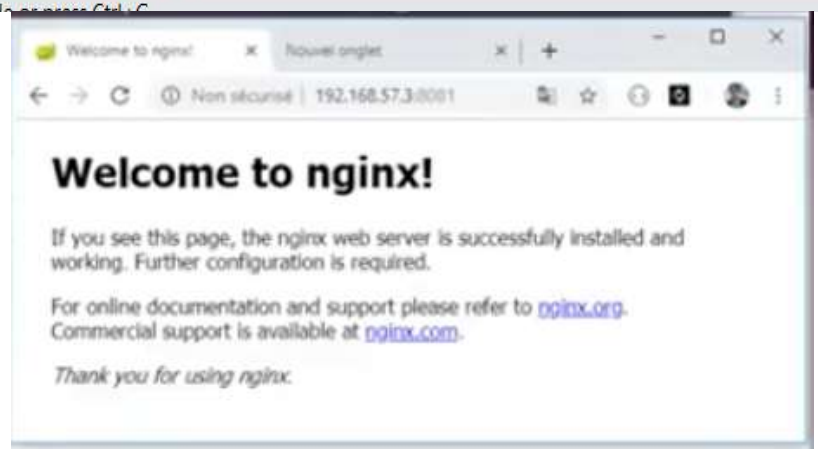
Conteneurisation avec Docker

Les commandes de base de docker : mapper les ports pour un accès externe

- Au démarrage nginx ouvre un service http sur le port 80
- Comme nginx démarre à l'intérieur du conteneur docker, l'accès http à cette application ne peut se faire qu'à l'intérieur du conteneur.
- Pour accéder à un service de l'extérieur à un service déployé à l'intérieur du conteneur, il faudrait qu'au démarrage du conteneur, spécifier à docker le mapping entre les ports exposés en interne avec les ports externes
- Cette opération se fait via l'option
  –p port externe:port interne
- Exemple:
- **$sudo docker run -d -p 8082:80 nginx**
- Dans notre cas : 192.168.57.3 représente l'adresse IP de la machine virtuelle ubuntu qui déploie docker Host




```
myubuntu@myubuntu:~$ sudo docker run -d -p 8082:80 nginx
ac0bac09d27510baa763c859b4cf6f7a36374ec0ac15bedc855b10d792f5b344
myubuntu@myubuntu:~$
```



Conteneurisation avec Docker

Les commandes de base de docker : images et rmi

- **$docker images**
  - Permet de lister les images présentées dans le host docker
- **$ docker rmi <<nom-image>>**
  - Permet de supprimer une image du host docker. Pour ce faire il faut s'assurer de stopper et supprimer d'abord tous les conteneurs instanciés de cette image

Conteneurisation avec Docker

Les commandes de base de docker : stop et rm

- Docker stop <<containerID or containerName>>
  - Permet de stopper un conteneur

```
myubuntu@myubuntu:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
8c3276761460   nginx     "/docker-entrypoint.…"   20 seconds ago   Up 19 seconds   0.0.0.0:8083->80/tcp, :::8083->80/tcp       amazing_haslett
ac0bac09d275   nginx     "/docker-entrypoint.…"   6 minutes ago    Up 6 minutes    0.0.0.0:8082->80/tcp, :::8082->80/tcp       serene_taussig
myubuntu@myubuntu:~$ docker stop 8c32
8c32
myubuntu@myubuntu:~$ docker stop ac0b
ac0b
```

- Docker rm << containerID or containerName>>
  - Permet de supprimer un conteneur d'une manière permanente

```
myubuntu@myubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
af2516a2a751   nginx     "/docker-entrypoint.…"   13 minutes ago   Up 13 minutes   0.0.0.0:8082->80/tcp, :::8082->80/tcp       dazzling_burnell
myubuntu@myubuntu:~$ docker rm af25
Error response from daemon: You cannot remove a running container af2516a2a75119498100a56138fc426a53ca4c7eb196b0fbffbc41528933c71e. Stop the container before attempting removal
 or force remove
myubuntu@myubuntu:~$ docker stop af25
af25
myubuntu@myubuntu:~$ docker rm af25
af25
```

Conteneurisation avec Docker

Un conteneur docker ne peut pas contenir un OS

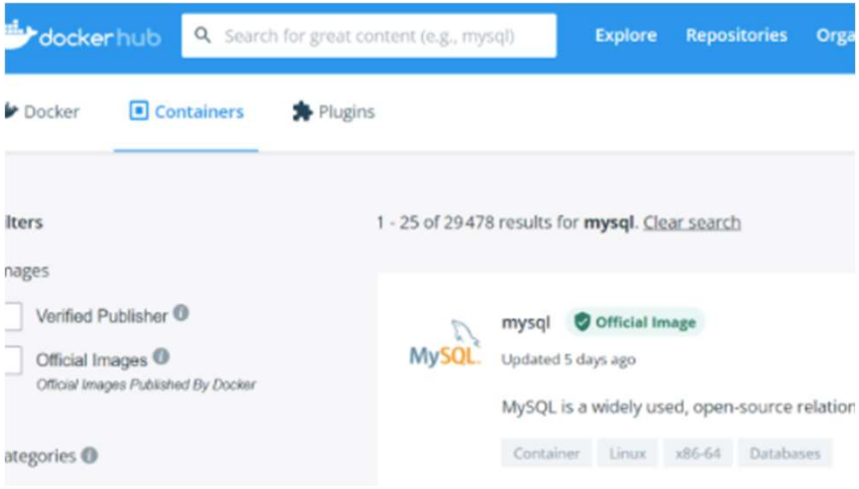
- Exemple : Tenter de démarrer un conteneur ubuntu
  - **$ docker run ubuntu**
- Le conteneur sera créé, mais pas démarré.
- En effet, un conteneur docker est fait pour contenir des processus et des services et non pas des OS
- L'image ubuntu est faites pour être l'image de base d'autres conteneurs et non pas l'exécuter en tant que telle
- Quand un service du conteneur s'arrête, le conteneur s'arrête aussi.

```
myubuntu@myubuntu: $ docker run ubuntu
myubuntu@myubuntu: $ docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                      PORTS     NAMES
c0240829ddef   ubuntu    "/bin/bash"   About a minute ago   Exited (0) About a minute ago             condescending_chatterjee
```

Conteneurisation avec Docker

La commande EXEC

- Permet d'exécuter une commande dans un conteneur démarré
- Exemple:
  - Démarrer un conteneur MySQL
  - Exécuter mysql dans le conteneur en mode interactif



docker run --name some-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -d mysql:tag

docker run --name my2 -e MYSQL_ROOT_PASSWORD=root -d mysql

Conteneurisation avec Docker

La commande EXEC

- docker exec -it my2 mysql --password
  - Cette ligne permet d'exécuter une commande mysql à l'intérieur d'un conteneur nommé mysql2, en demandant le mdp
  - on peut se connecter au serveur mysql et taper les cmds correspondants. Les résultats seront envoyés sur le terminal puisque le mode itératif -it est défini

```
myubuntu@myubuntu:~$ docker run --name mysql2 -e MYSQL_ROOT_PASSWORD=root -d mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
8e0176adc18c: Pull complete
14e977b0f4b2: Pull complete
a7b58dd6f78b: Pull complete
fba70cc872a5: Pull complete
5db2cc6eab8f: Pull complete
081f41f573ba: Pull complete
86bf2dc4ded9: Pull complete
47f08b0e916e: Pull complete
850e29ae8eeb: Pull complete
13517fe0d921: Pull complete
Digest: sha256:f61944ff3f2961363a4d22913b2ac581523273679d7e14dd26e8db8c9f571a7e
Status: Downloaded newer image for mysql:latest
2ba7fe19f14ed11e9e90fa141571612b234faf65629a5f9b7a39abf965939ad5
myubuntu@myubuntu:~$ docker exec -it  mysql2 mysql --password
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)
```

Conteneurisation avec Docker

La commande Attach

- Quand vous lancez un conteneur comme tâche de fond avec l'options -d, il se peut que vous ayez besoin d'attacher la console du terminal courant au conteneur.
- Cette opération se fait via la commande :
  - $ docker attach containerID
- Exemple:




```
C:\WINDOWS\system32>docker run -d alpine ping 8.8.8.8
f486e756d97bb9c887306acedd56615c7d63d10c25e9b44681b5781b699e9ad7

C:\WINDOWS\system32>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS              PORTS
                                                                                    NAMES
f486e756d97b   alpine                 "ping 8.8.8.8"           6 seconds ago        Up 4 seconds
C:\WINDOWS\system32>docker attach f486e756d97b
```

Conteneurisation avec Docker

Inspecter un conteneur

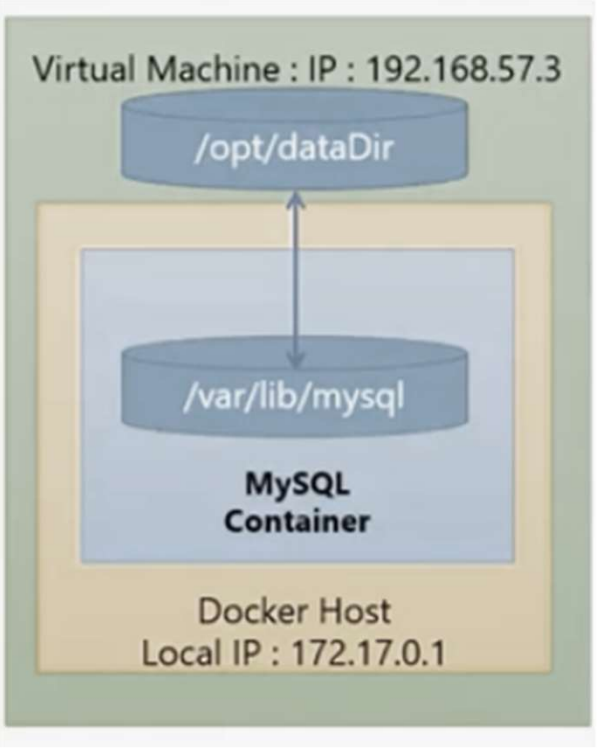
- La commande ps permet d'avoir des informations basiques sur les conteneurs
- Pour avoir plus de détail sur les conteneurs au format JSON, on peut utiliser la commande:
- $docker inspect <<container_name or container_id>>

```
myubuntu@myubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                  NAMES
2ba7fe19f14e   mysql     "docker-entrypoint.s…"   26 hours ago   Up 26 hours   3306/tcp, 33060/tcp   mysql2
myubuntu@myubuntu:~$ docker inspect 2ba7
[
    {
        "Id": "2ba7fe19f14ed11e9e90fa141571612b234faf65629a5f9b7a39abf965939ad5",
        "Created": "2023-10-23T10:21:56.164794667Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "mysqld"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
```

Conteneurisation avec Docker

Mapping des volumes: persistances des données dans un conteneur

- Quand vous démarrez un conteneur MySQL, les données des bases de données sont stockées par défaut dans le dossier **/var/lib/mysql** à l'intérieur du docker container.
- Un docker container dispose de son propre système de fichier.
- Si vous supprimer ce conteneur MySQL, toutes les bases de données seront détruites.
- Vous vous souhaitez rendre les données persistantes en dehors du conteneur, il faudrait mapper les volume au démarrage du conteneur.
- **$ docker run -v /opt/dataDir: /var/lib/mysql** mysql
- Quand le conteneur mysql démarre, il monte implicitement un dossier en dehors du conteneur vers un dossier interne au conteneur.
- Dans ce cas , toutes les données seront stockées dans un dossier externe au conteneur et même si on supprime le conteneur, les données restent dans le dossier /opt/dataDir



Conteneurisation avec Docker

Logs des conteneurs

- Quand vous démarrez un conteneur comme tâche de fond, il serait utile un moment donné de visualiser le fichier log d'un conteneur.
- Cette opération peut se faire par la commande:
- **$ docker logs container_ID**

```
myubuntu@myubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                 NAMES
2ba7fe19f14e   mysql     "docker-entrypoint.s…"   26 hours ago   Up 26 hours   3306/tcp, 33060/tcp   mysql2
myubuntu@myubuntu:~$ docker logs 2ba7
2023-10-23 10:21:56+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.1.0-1.el8 started.
2023-10-23 10:21:57+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2023-10-23 10:21:57+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.1.0-1.el8 started.
2023-10-23 10:21:57+00:00 [Note] [Entrypoint]: Initializing database files
2023-10-23T10:21:57.247196Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2023-10-23T10:21:57.249186Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_ca
che_size=0 instead.
2023-10-23T10:21:57.249275Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.1.0) initializing of server in progress as process 80
2023-10-23T10:21:57.253870Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2023-10-23T10:21:57.756942Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2023-10-23T10:21:58.462491Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
```

Conteneurisation avec Docker

Dockerfile

- Un Dockerfile est un fichier texte qui contient les commandes nécessaires à la construction d'une image.
- Une fois qu'un Dockerfile est créé, l'administrateur utilise la commande docker build pour créer une image basée sur les commandes contenues dans le fichier.
- Les commandes et les informations contenues dans le Dockerfile peuvent être configurées pour utiliser des versions et des dépendances logicielles spécifiques afin de garantir des déploiements cohérents et stables.

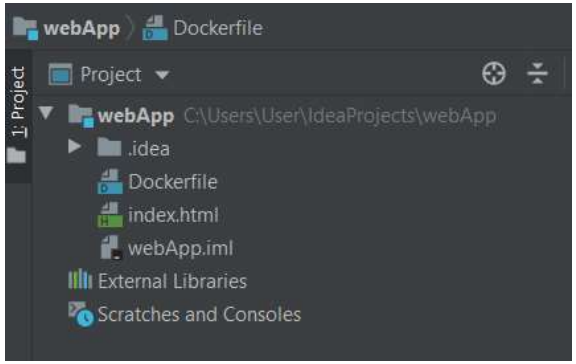
Conteneurisation avec Docker

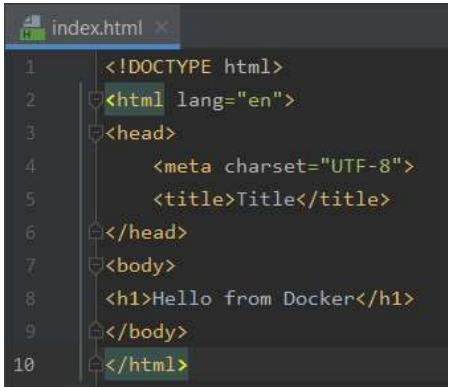
Dockerfile

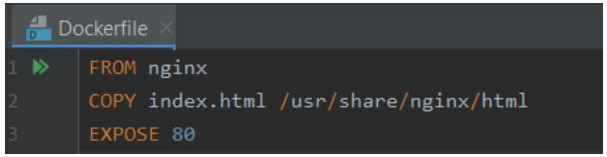
- Un Dockerfile utilise les commandes suivantes pour construire les images :
  - **COPY :** Copie des fichiers ou dossiers locaux depuis le contexte de build vers l'image Docker.
  - **ADD :** Copie des fichiers ou dossiers et peut aussi extraire des archives ou télécharger des fichiers depuis une URL.
  - **CMD** : Exécuter une commande spécifique dans le conteneur.
  - **RUN** : Directive d'exécution centrale pour les fichiers Docker.
  - **ENTRYPOINT** : Définit le programme ou script qui sera toujours exécuté quand le conteneur démarre, en acceptant éventuellement des arguments supplémentaires.
  - **ENV** : Définir des variables d'environnement.
  - **EXPOSE** : Exposer un port spécifique pour permettre la mise en réseau du conteneur avec le monde extérieur.
  - **FROM** : Définir l'image de base utilisée pour démarrer le processus de construction.
  - **MAINTAINER / LABEL** : Définit le nom complet et l'adresse électronique du créateur de l'image.
  - **USER** : Définir l'UID (le nom d'utilisateur) qui exécutera le conteneur.
  - **VOLUME** : Déclare un dossier qui doit être stocké de façon persistante (en dehors du conteneur).
  - **WORKDIR** : Définir le chemin où la commande, définie avec CMD, doit être exécutée.

Conteneurisation avec Docker

Dockerfile







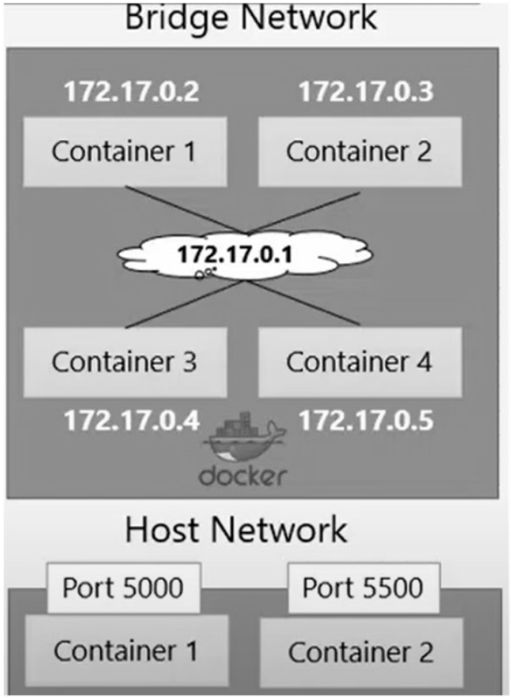

```
C:\Users\User\IdeaProjects\webApp>docker build -t myapp .
```

```
C:\Users\User\IdeaProjects\webApp>docker run -d -p 8089:80 myapp
2e05049188f293b93a1da72a0e3e817c2eabe74ab403a05a5dae5012e1e14a5e
```

Conteneurisation avec Docker

Networking dans Docker

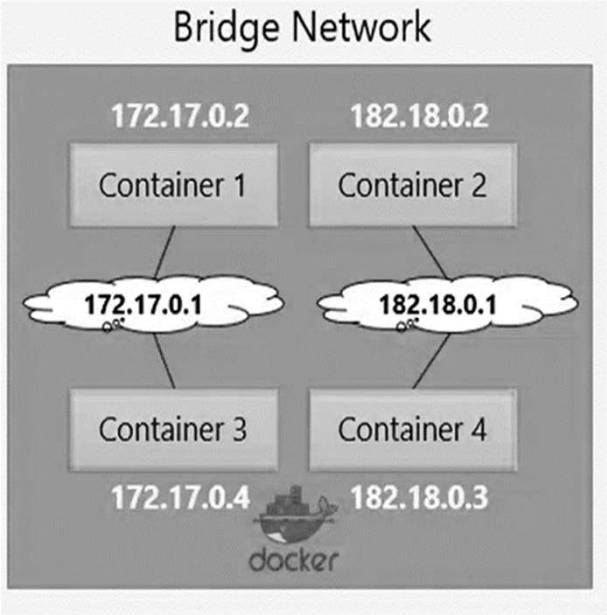
- Il existe 3 modes réseaux en docker :
  - Bridge : Par défaut : $ docker run nginx
  - Host: $ docker run --network=host nginx
  - none: $ docker run --network=none nginx
- Bridge est le mode réseau par défaut qui est utilisé par conteneur exécuté.
  - C'est le réseau privé interne créé par docker dans le Host.
  - Tous les conteneurs sont attachés par défaut à ce réseau
  - Chaque conteneur a une adresse souvent 172.17.0.x
  - Chaque conteneur peut accéder à l'autre par cette adresse.
  - Pour accéder à l'un des ces conteneur de l'extérieur, il faut mapper le port de ces conteneurs aux ports du docker host.
- Une autre manière pour accéder aux conteneurs de l'extérieur sans mapper les ports c'est de les attacher au réseau de type host.
  - Dans ce cas le port de chaque conteneur est mappé automatiquement au même port au niveau du host.
  - Ce qui fait qu'on ne pourra pas démarrer plusieurs instances du même conteneur, car on ne peut pas exposer plusieurs services avec le même port au niveau du host.
- Le troisième type de réseaux est none, signifie que le conteneur n'est attaché à aucun réseau. Il ne sera donc pas accessible ni localement ni de l'extérieur.



Conteneurisation avec Docker

Networking dans Docker

- Dans le cas du réseau de type Bridge, il est possible de connecter les conteneurs dans des réseaux internes différents :
- Pour cela créer un nouveau réseau :
  - $ docker network create --driver bridge –subnet 182.18.0.0/16 custom-isolated-net
- Pour lister les réseaux disponibles :
  - $ docker network ls
- Pour consulter la configuration réseau d'un conteneur
  - $ docker inspect <<container_id or container name>>



Conteneurisation avec Docker

Networking dans Docker

```
C:\WINDOWS\system32>docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
2b28d98689c8   bridge    bridge    local
4b012da2f342   host      host      local
69075739c1e0   none      null      local
```

```
C:\WINDOWS\system32>docker network create --driver bridge --subnet 172.19.0.0/16 myNet
99ac327cb052e8d58f5cc3c086b71d563720c6c6e1f3ccafe06f20f5955b17fb

C:\WINDOWS\system32>docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
3ad9b9f4d176   bridge    bridge    local
4b012da2f342   host      host      local
99ac327cb052   myNet     bridge    local
69075739c1e0   none      null      local
```

```
C:\WINDOWS\system32>docker network inspect 99ac
[
    {
        "Name": "myNet",
        "Id": "99ac327cb052e8d58f5cc3c086b71d563720c6c6e1f3ccafe06f20f5955b17fb",
        "Created": "2023-11-20T10:58:54.6102089Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {},
        "Labels": {}
    }
]
```

Conteneurisation avec Docker

Networking dans Docker

```
C:\WINDOWS\system32>docker run -d --network=myNet nginx
63fa1c946f43eb5d0b0709b37b725fcacab29c2d15929836325098ec431c2043

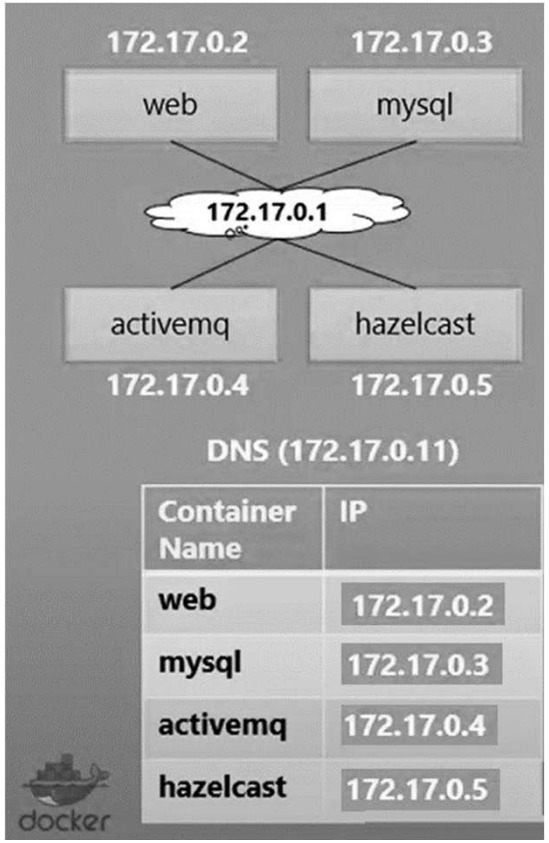
C:\WINDOWS\system32>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NA
ES
63fa1c946f43   nginx     "/docker-entrypoint.…"   11 seconds ago   Up 11 seconds   80/tcp    cl
ver_lumiere

C:\WINDOWS\system32>docker inspect 63fa
[
    {
        "Id": "63fa1c946f43eb5d0b0709b37b725fcacab29c2d15929836325098ec431c2043",
        "Created": "2023-11-20T11:09:44.009698835Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
            "-g",
            "daemon off;"
        ],
                    "NetworkID": "99ac327cb052e8d58f5cc3c086b71d563720c6c6e1f3ccafe06f20f5955b17
fb",
                    "EndpointID": "3deeacab8164fbfdd54f402f6006500bc532311b36106442a8f8d3ecf4ff7
17a",
                    "Gateway": "172.19.0.1",
                    "IPAddress": "172.19.0.2",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "MacAddress": "02:42:ac:13:00:02",
                    "DriverOpts": null
```

Conteneurisation avec Docker

Docker Name Spaces (DNS)

- Un conteneur peut atteindre un autre conteneur par son nom
- Docker utiliser DNS (Docker Name Spaces) pour associer l'adresse IP au nom du conteneur.
- DNS est un serveur qui est démarré par défaut en utilisant l'adresse IP 172.17.0.11
- Il est plus pratique de connecter les conteneurs en utilisant leurs noms, car les adresses IP peuvent changer au démarrage.



Conteneurisation avec Docker

Stockage dans Docker

- Quand vous installer Docker dans un système de fichier, il y crée son dossier dans :
- /var/lib/docker
  - aufs
  - containers
  - image
  - Volumes
- C'est dans ces dossiers où docker stocke, par défaut, toutes ses données. C'est à dire les fichiers relatifs aux images, conteneurs, volumes.

Conteneurisation avec Docker

Architecture en couches

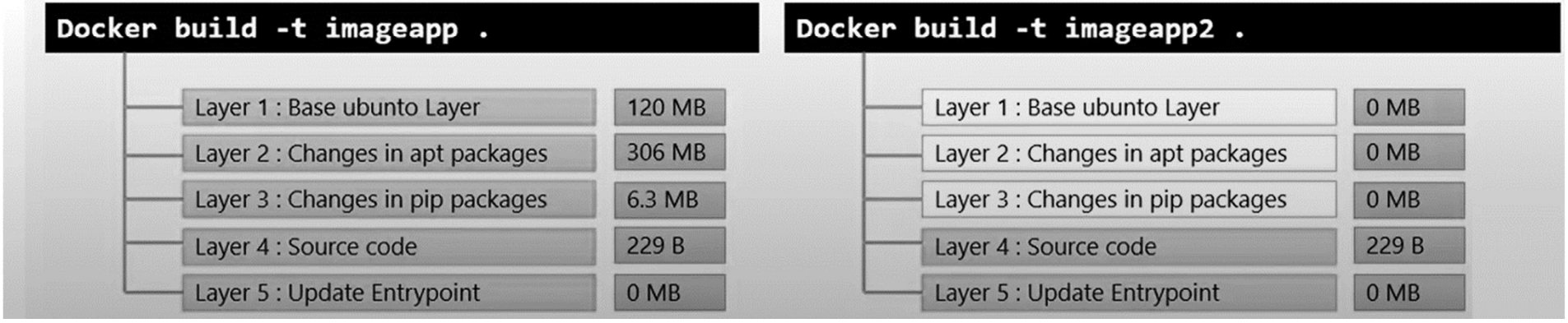
- Pour comprendre comment docker gère ses données, considérant les deux exemples suivants de fichier docker.

Dockerfile1

```
FROM ubuntu
RUN apt-get update && apt-get -y install python
RUN pip install flusk flusk-mysql
COPY . /opt/source-code
ENTRYPOINT FLUSK_APP=/opt/source-code/app.py flusk run
```

Dockerfile2

```
FROM ubuntu
RUN apt-get update && apt-get -y install python
RUN pip install flusk flusk-mysql
COPY app2.py /opt/source-code
ENTRYPOINT FLUSK_APP=/opt/source-code/app2.py flusk run
```



Conteneurisation avec Docker

Docker Compose

- Compose est un outil permettant de définir et d'exécuter des applications Docker à conteneurs multiples.
- Avec Compose, vous utilisez un fichier au format YAML (docker-compose.yml) pour configurer les services de votre application.
- Ensuite, avec une seule commande, vous créez et démarrez tous les services de votre configuration.
  - $ docker-compose up

```
$ docker run --name=web simple-web-app
$ docker run --name=database mongodb
$ docker run --name=messaging redis:alpine
$ docker run --name=orchestration ansible
```

docker-compose.yml

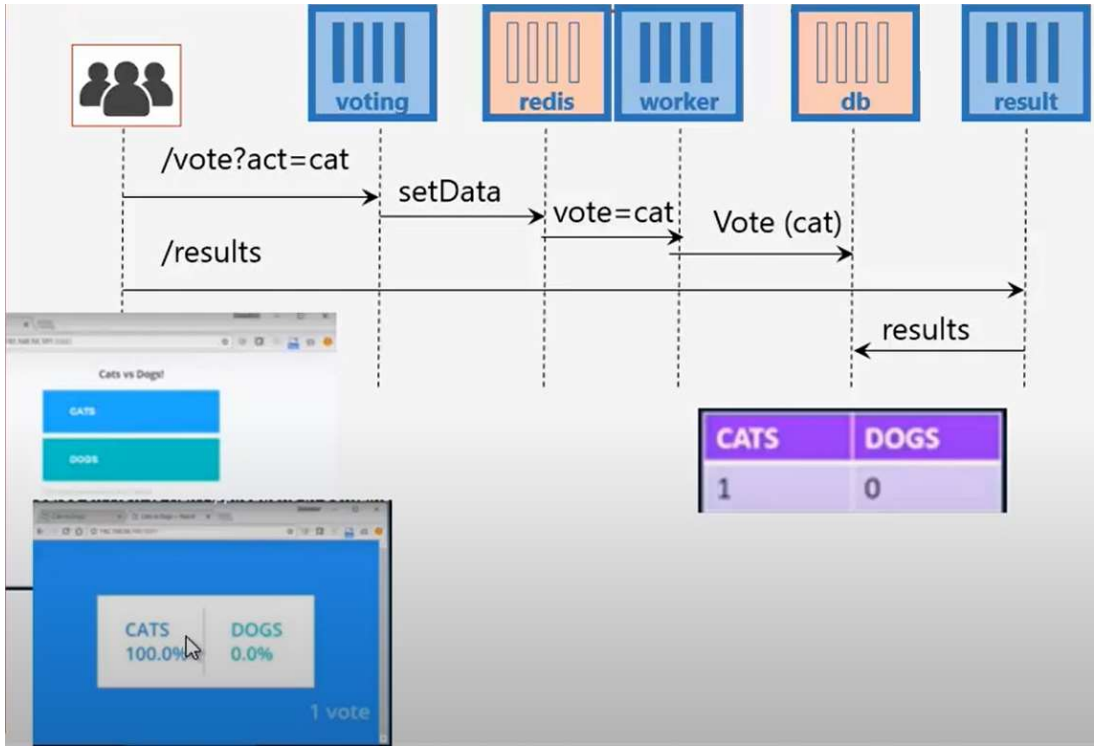
```
services:
   web:
      image: "simple-web-app"
   database:
      image:"mongodb"
   messaging:
      image:"redis"
   orchestration:
      image:"ansible"
```

```
$ docker-compose up
```

Conteneurisation avec Docker

Docker Compose: exemple d'application

- Considérons un exemple fourni dans la documentation de Docker
- Cette application permet de voter chats ou chiens et d'afficher les résultats du vote.
- L'appli se compose de 5 services :
  - Voting-app : web application développée en python
  - Redis : Cache mémoire
  - Worker : développé en Java
  - Postgresq| : pour stocker les données du vote
  - Result-app : pour présenter les résultats.

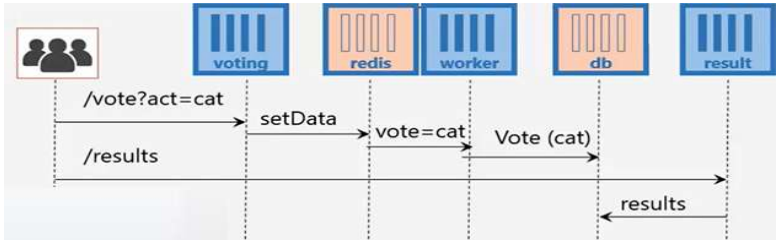


Conteneurisation avec Docker

Docker Compose: exemple d'application

- On peut tenter de démarrer manuellement les services avec la commande run
- En utilisant l'option --link pour lier les services
- Cette manière est trop lourde
- Il est plus simple de faire la même chose avec docker-compose

https://github.com/eesprit/example-voting-app




```
class Worker {
  public static void main(String[] args) {
    try {
      Jedis redis = connectToRedis("redis");
      Connection dbConn = connectToDB("db");
```

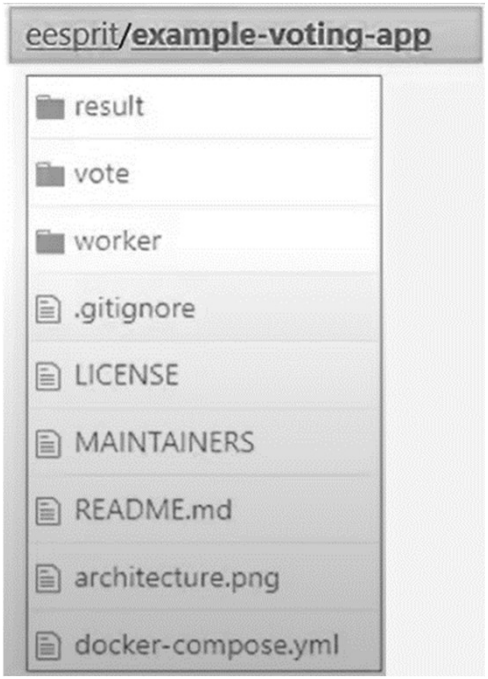
```
$ docker run -d --name=redis redis
$ docker run -d --name=db postgres:9.4
$ docker run -d --name=vote –p 5000:80  --link=redis:redis  /eesprit/voting-app-vote
$ docker run -d --name=result –p 5001:80 --link=db:db result-app
$ docker run -d --name=worker --link=redis:redis --link=db:db worker
```

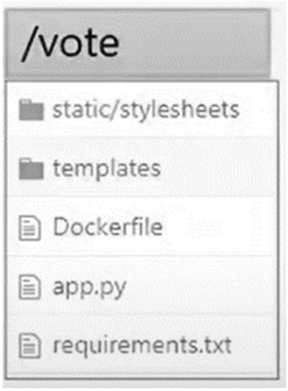
```
def get_redis():
    if not hasattr(g, 'redis'):
        g.redis = Redis(host="redis", db=0, socket_timeout=5)
    return g.redis
```

```
async.retry(
  {times: 1000, interval: 1000},
  function(callback) {
    pg.connect('postgres://postgres@db/postgres', function(err, client, done) {
```

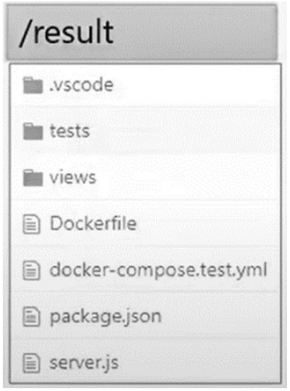
Conteneurisation avec Docker

Docker Compose: déploiement de l'application









- Dans la version 2 de docker-compose
  - L'option links n'est pas obligatoire
  - Docker crée automatiquement un lien entre tous les services déclarés dans docker-compose

```
$ docker-compose up
```

docker-compose.yml

```
version: "2"
services:
  vote:
    image: eesprit/voting-app-vote
    ports:
    - "5000:80"
    links:
    - redis
  redis:
    image: redis:alpine
  worker:
    image: eesprit/voting-app-worker
    links:
    - redis
    - db
  db:
    image: postgres:9.4
  result:
    image: eesprit/voting-app-result
    links:
    - db
    ports:
    - "5001:80"
```

Conteneurisation avec Docker

Docker Compose: si les images ne sont pas créées

Vote/Dockerfile

```
# Using official python runtime base image
FROM python:2.7-alpine

# Set the application directory
WORKDIR /app

# Install our requirements.txt
ADD requirements.txt /app/requirements.txt
RUN pip install -r requirements.txt

# Copy our code from the current folder to /app inside the container
ADD . /app

# Make port 80 available for links and/or publish
EXPOSE 80

# Define our command to be run when launching the container
CMD ["gunicorn", "app:app", "-b", "0.0.0.0:80", "--log-file", "-", "--access-
logfile", "-", "--workers", "4", "--keep-alive", "0"]
```

Vote/requirements.txt

```
Flask
Redis
gunicorn
```

docker-compose.yml

```
version: "2"
services:
  vote:
    build: ./vote
    command: python app.py
    volumes:
    - ./vote:/app
    ports:
    - "5000:80"
  redis:
    image: redis:alpine
    ports: ["6379"]
  worker:
    build: ./worker
  db:
    image: postgres:9.4
  result:
    build: ./result
    command: nodemon --debug server.js
    volumes:
    - ./result:/app
    ports:
    - "5001:80"
    - "5858:5858"
```

Conteneurisation avec Docker

Docker Compose: si les images ne sont pas créées

worker/Dockerfile

```
FROM java:openjdk-8-jdk

RUN MAVEN_VERSION=3.3.3 \
 && cd /usr/share \
 && wget http://archive.apache.org/dist/maven/maven-3/$MAVEN_VERSION/binaries/apache-
maven-$MAVEN_VERSION-bin.tar.gz -O - | tar xzf - \
 && mv /usr/share/apache-maven-$MAVEN_VERSION /usr/share/maven \
 && ln -s /usr/share/maven/bin/mvn /usr/bin/mvn

WORKDIR /code

ADD pom.xml /code/pom.xml
RUN ["mvn", "dependency:resolve"]
RUN ["mvn", "verify"]

# Adding source, compile and package into a fat jar
ADD src /code/src
RUN ["mvn", "package"]

CMD ["java", "-jar", "target/worker-jar-with-dependencies.jar"]
```

docker-compose.yml

```
version: "2"
services:
  vote:
    build: ./vote
    command: python app.py
    volumes:
    - ./vote:/app
    ports:
    - "5000:80"
  redis:
    image: redis:alpine
    ports: ["6379"]
  worker:
    build: ./worker
  db:
    image: postgres:9.4
  result:
    build: ./result
    command: nodemon --debug server.js
    volumes:
    - ./result:/app
    ports:
    - "5001:80"
    - "5858:5858"
```

Conteneurisation avec Docker

Docker Compose: si les images ne sont pas créées

result/Dockerfile

```
FROM node:5.11.0-slim

WORKDIR /app

RUN npm install -g nodemon
ADD package.json /app/package.json
RUN npm config set registry http://registry.npmjs.org
RUN npm install && npm ls
RUN mv /app/node_modules /node_modules

ADD . /app

ENV PORT 80
EXPOSE 80

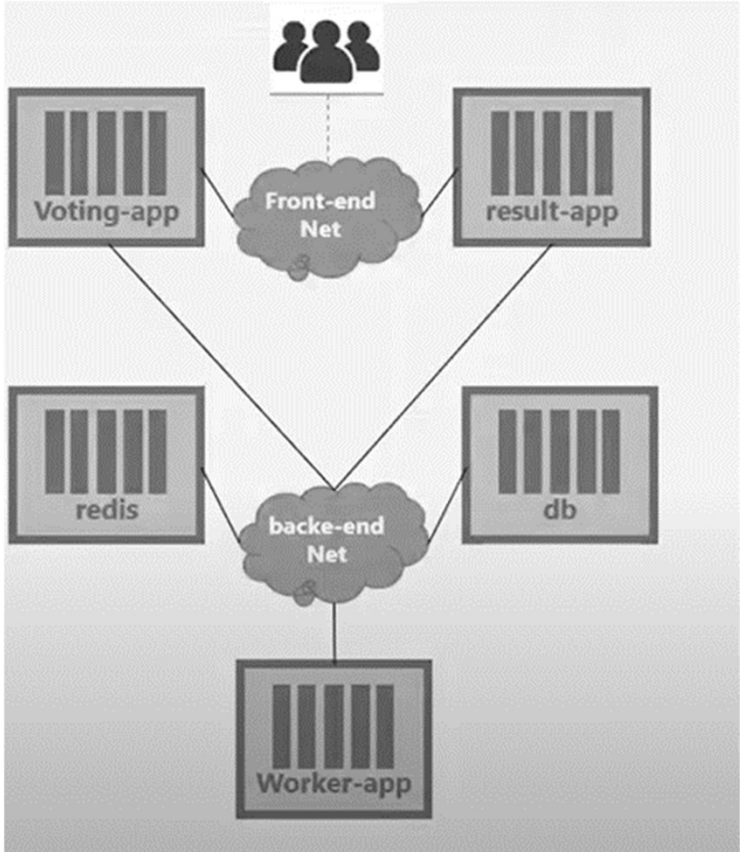
CMD ["node", "server.js"]
```

docker-compose.yml

```
version: "2"
services:
  vote:
    build: ./vote
    command: python app.py
    volumes:
    - ./vote:/app
    ports:
    - "5000:80"
  redis:
    image: redis:alpine
    ports: ["6379"]
  worker:
    build: ./worker
  db:
    image: postgres:9.4
  result:
    build: ./result
    command: nodemon --debug server.js
    volumes:
    - ./result:/app
    ports:
    - "5001:80"
    - "5858:5858"
```

Conteneurisation avec Docker

Docker Compose: Networking



docker-compose.yml

```
version: "2"
services:
  vote:
    image: eesprit/voting-app-vote
    ports:
    - "5000:80"
    networks:
     - front-end
     - back-end
  redis:
    image: redis:alpine
    networks:
     - back-end
  worker:
    image: eesprit/voting-app-worker
    networks:
     - back-end
  db:
    image: postgres:9.4
    networks:
     - back-end
  result:
    image: eesprit/voting-app-result
    ports:
    - "5001:80«
  networks:
     - front-end
     - back-end
networks:
    front-end :
    back-end :
```

Conteneurisation avec Docker

Docker Registry

- Les images docker publiques sont stockées dans le cloud :
  - Docker Hub (docker.io)
- Quand vous exécuter un conteneur nginx avec la commande :
  - $ docker run nginx
  - Nginx représente le nom de l'image dans un repository.
- Ici le repository est nginx qui représente le nom de l'utilisateur docker hub.
  - $ docker run nginx/nginx
  - $ docker run user_name/image_name

Conteneurisation avec Docker

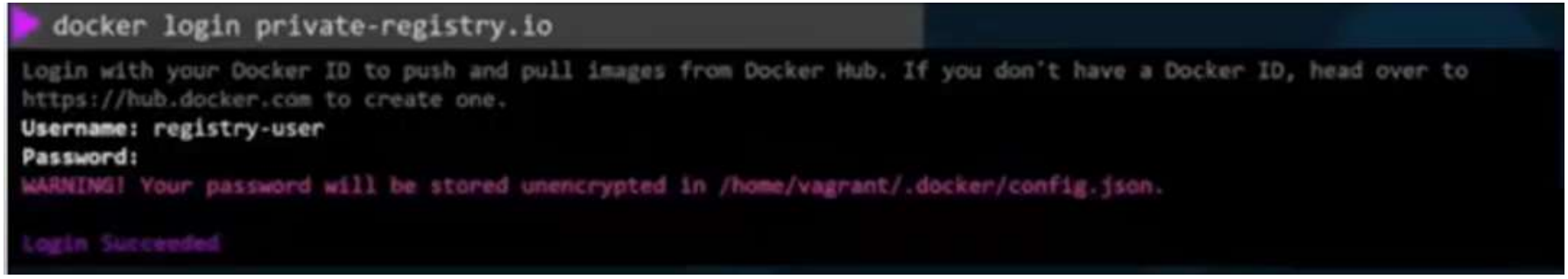
Docker Registry

- Docker registry est lui-même une application exposée dans image docker disponible sur docker hub.
- Vous pouvez installer dans le host du docker un annuaire privé de docker :
  - $ docker run -d -p 5000:5000 --name my-registry registry:2
- Pour pousser et récupérer une image docker dans ce nouveau registre interne
  - $ docker image tag my-image localhost:5000/my-image
  - $ docker push localhost:5000/my-image
  - $ docker pull localhost:5000/my-image

Conteneurisation avec Docker

Docker Registry

- Les organisations peuvent créer leur annuaire interne
- Dans ce cas là, l'accès passe une opération d'authentification avec:
  - $ docker login private-registry.io
- Pour accéder à une image:
  - $ docker run private-registry.io/apps/internal-app



Conteneurisation avec Docker

Restriction des ressources avec cgroups

- Par défaut un conteneur peut utiliser toutes les ressources disponibles (CPUs et RAM) dans le host Linux
- Pour restreindre l'utilisation des ressources pour un conteneur:
  - $ docker run --cpus=0.5 ngninx
  - $ docker run --memory=100m mysql

